
Modül 21 — gRPC

Ders Notları

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 gRPC Nedir	2
2.2 REST ve gRPC Karşılaştırması	2
2.3 Dört RPC Türü	3
3. Adım Adım Örnekler	3
3.1 Sözleşme ve Kod Üretimi	4
3.2 Bir Unary Servis	5
3.3 Streaming	6
3.4 Hatalar: Durum Kodları	8
3.5 Interceptor'lar	8
3.6 İstemciler ve Deadline'lar	9
3.7 Health, Reflection ve Testler	11
4. Sık Yapılan Hatalar ve En İyi Pratikler	12
5. Özet	12
6. İleri Okuma	12

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Protocol Buffers ile bir servis sözleşmesi yazmak ve ondan Maven ile Java kodu üretmek
- Unary, server streaming, client streaming ve çift yönlü streaming RPC'leri @GrpcService ile gerçekleştirmek
- Java exception'larını @GrpcAdvice ile gRPC durum kodlarına eşlemek
- Bir sunucu interceptor'ı eklemek ve servisleri deadline'lı client stub'ları üzerinden çağırmak
- Standart health ve reflection servislerini kullanmak ve her şeyi in-process test etmek
- Belirli bir API için REST ile gRPC arasında karar vermek

Ön koşullar: Modül 03 (REST API'leri) · **Tahmini süre:** 4 saat · **Docker gerekmez**

2. Kavramlar

2.1 gRPC Nedir

gRPC bir uzak prosedür çağrısı (RPC) çatısıdır. İstemci üretilmiş bir **stub** üzerindeki bir metodu yerel bir nesneymiş gibi çağırır ve çağrı HTTP/2 üzerinden sunucuya gider. Sözleşme bir .proto dosyasıdır. Mesajlar **Protocol Buffers** olarak kodlanır: Her alanın adıyla değil **numarasıyla** tanımlandığı kompakt, ikili bir format.

2.2 REST ve gRPC Karşılaştırması

	REST (HTTP üzerinden JSON)	gRPC (HTTP/2 üzerinden Protobuf)
Sözleşme	isteğe bağlı (OpenAPI), çoğu zaman koddan sonra yazılır	zorunlu, .proto dosyası önce gelir
Veri	okunabilir JSON metni	ikili, küçük ve hızlı ayrıştırılır
Kod	elle yazılmış istemci veya OpenAPI'den üretilmiş	birçok dil için üretilmiş istemci ve sunucu stub'ları
Streaming	sınırlı (SSE, WebSocket)	yerleşik: istemci, sunucu ve çift yönlü

	REST (HTTP üzerinden JSON)	gRPC (HTTP/2 üzerinden Protobuf)
Hatalar	HTTP durumu + gövde (ProblemDetail)	16 durum kodu + açıklama + metadata
Tarayıcılar	doğrudan çalışır	bir proxy gerekir (gRPC-Web)
Araçlar	curl, tarayıcı	grpcurl, Postman, reflection
Uygun olduğu yer	public API'ler, web istemcileri, önbellekleme	servisler arası iç çağrılar, streaming, çok dil

Bitirme projesinde gateway dışarıya REST sunar ve iç servisler gRPC konuşur.

2.3 Dört RPC Türü

Tür	İstek	Yanıt	Bu dersteki örnek
Unary	bir	bir	GetBook
Server streaming	bir	çok	ListBooks
Client streaming	çok	bir	ImportBooks
Çift yönlü streaming	çok	çok, istenen an	Chat (stok soruları ve cevapları)

3. Adım Adım Örnekler

Uygulamayı başlatın. gRPC sunucusu 9090 portunu dinler ve tur onu bir client stub üzerinden çağırır:

```
./mvnw -pl modules/21-grpc/lesson spring-boot:run
```

```
=== 3.2 Unary: GetBook ===
Effective Java by Joshua Bloch, 8990 cents, 48 bytes on the wire
=== 3.3 Server streaming: ListBooks ===
Effective Java
Java Puzzlers
...
=== 3.4 Errors are status codes ===
NOT_FOUND: No book with ISBN 00000000000000
```

Aynı kitap kompakt JSON olarak 91 bayttır: Alan adları her mesajla birlikte gider. Reflection açık olduğunda [grpcurl](#), sunucuyu .proto dosyası olmadan çağırabilir (bkz. README).

i Not

JDK 27’de protobuf-java, sun.misc.Unsafe hakkında bir uyarı yazdırır. Şimdilik zararsızdır. --sun-misc-unsafe-memory-access=allow JVM seçeneği, protobuf-java yeni bellek API’lerine geçene kadar onu gizler.

3.1 Sözleşme ve Kod Üretimi

```
syntax = "proto3";

package bookstore.catalog.v1;

option java_package = "com.springbootedu.grpc.catalog.v1";
option java_multiple_files = true;

service BookCatalog {
  rpc GetBook (GetBookRequest) returns (Book);           // unary
  rpc ListBooks (ListBooksRequest) returns (stream Book); // server streaming
  rpc ImportBooks (stream Book) returns (ImportSummary); // client streaming
  rpc Chat (stream StockQuestion) returns (stream StockAnswer); // bidirectional
    ⇨ streaming
}

message GetBookRequest {
  string isbn = 1;
}

message ListBooksRequest {
  string author = 1;           // empty: all books
}

message Book {
  string isbn = 1;
  string title = 2;
  string author = 3;
  int64 price_cents = 4;       // money as whole cents: no floating point errors
}

message ImportSummary {
  int32 imported = 1;
  int32 rejected = 2;
}
```

```
message StockQuestion {
    string isbn = 1;
}

message StockAnswer {
    string isbn = 1;
    int32 available = 2;
}
```

- package gRPC adıdır (bookstore.catalog.v1.BookCatalog/GetBook), java_package üretilen kodun Java paketidir.
- Bir tipin önündeki stream, o tarafı bir akış yapar.
- Para, sent cinsinden bir tamsayıdır: double yuvarlama hataları getirirdi.

Spring Boot, ascpes'un protobuf-maven-plugin'ini yönetir ve onu spring-boot-starter-parent üzerinden yapılandırır. Eklenti src/main/proto'yu okur ve mesajları ve BookCatalogGrpc sınıfını target/generated-sources/protobuf'a yazar:

```
<!-- generates Java messages and gRPC stubs from src/main/proto (configured by
↳ spring-boot-starter-parent) -->
<plugin>
  <groupId>io.github.ascpes</groupId>
  <artifactId>protobuf-maven-plugin</artifactId>
</plugin>
```

3.2 Bir Unary Servis

Bir servis üretilmiş ...ImplBase sınıfını genişletir. @GrpcService onu gRPC sunucusunun yayınladığı bir bean yapar:

```
@Override
public void getBook(GetBookRequest request, StreamObserver<Book> responses) {
    if (!request.getIsbn().matches("\\d{13}")) {
        throw new IllegalArgumentException("An ISBN has 13 digits: " +
            ↳ request.getIsbn()); // → INVALID_ARGUMENT
    }
    Book book = store.find(request.getIsbn()).orElseThrow(() -> new
        ↳ BookNotFoundException(request.getIsbn()));
    responses.onNext(book); // exactly one answer ...
    responses.onCompleted(); // ... and the call is over
}
```

Yanıt bir StreamObserver'a gider: onNext bir mesaj gönderir, onCompleted çağrısı başarıyla bitirir, onError onu bir durumla bitirir.

3.3 Streaming

Server streaming: Sunucu, sonucu olduğu kadar onNext çağırır:

```
@Override
public void listBooks(ListBooksRequest request, StreamObserver<Book> responses) {
    store.byAuthor(request.getAuthor()).forEach(responses::onNext);    // many answers,
    ↪ one after the other
    responses.onCompleted();
}
```

Client streaming: Metot, istemcinin mesajlarını alan bir observer döndürür. Yanıt, istemci onCompleted çağırıldığında gelir:

```
@Override
public StreamObserver<Book> importBooks(StreamObserver<ImportSummary> responses) {
    return new StreamObserver<>() {
        ↪ for the client's stream
        private int imported;
        private int rejected;

        @Override
        public void onNext(Book book) {
            if (book.getIsbn().matches("\\d{13}")) {
                store.add(book, 1);
                imported++;
            } else {
                rejected++;
            }
        }

        @Override
        public void onError(Throwable error) {
            // the client cancelled or the connection broke: nothing to answer
        }

        @Override
        public void onCompleted() {
            ↪ one summary
            ↪ respon-
            ses.onNext(ImportSummary.newBuilder().setImported(imported).setRejected(rejected).build());
            responses.onCompleted();
        }
    };
}
```

Çift yönlü streaming: İki taraf da istediği zaman gönderir ve akış ikisi de bitene kadar

açık kalır:

```
@Override
public StreamObserver<StockQuestion> chat(StreamObserver<StockAnswer> responses) {
    return new StreamObserver<>() {
        @Override
        public void onNext(StockQuestion question) { // answer each question at once,
            ↪ the stream stays open
            responses.onNext(StockAnswer.newBuilder()
                .setIsbn(question.getIsbn())
                .setAvailable(store.stockOf(question.getIsbn()))
                .build());
        }

        @Override
        public void onError(Throwable error) {
        }

        @Override
        public void onCompleted() {
            responses.onCompleted();
        }
    };
}
```

İstemci tarafında akışlar asenkron stub'a (BookCatalogStub) ihtiyaç duyar. Test, import akışına üç kitap gönderir ve bir özet alır:

```
@Test
void clientStreaming() throws Exception {
    var summary = new CompletableFuture<ImportSummary>();
    StreamObserver<Book> upload = asyncCatalog.importBooks(new StreamObserver<>() {
        @Override
        public void onNext(ImportSummary value) {
            summary.complete(value);
        }

        @Override
        public void onError(Throwable error) {
            summary.completeExceptionally(error);
        }

        @Override
        public void onCompleted() {
        }
    });
}
```

```

upload.onNext(book("9781098150358", "Learning Spring Boot 3.0"));
upload.onNext(book("123", "Broken ISBN"));
upload.onNext(book("9781617294945", "Spring Microservices in Action"));
upload.onCompleted(); // the client says: no more books

ImportSummary result = summary.get(5, TimeUnit.SECONDS);
assertThat(result.getImported()).isEqualTo(2);
assertThat(result.getRejected()).isEqualTo(1);
}

```

3.4 Hatalar: Durum Kodları

gRPC'nin kendi durum kodları vardır: OK, NOT_FOUND, INVALID_ARGUMENT, DEADLINE_EXCEEDED, UNAVAILABLE, PERMISSION_DENIED, ... Eşlemesi olmayan bir exception istemciye kullanışlı bir mesaj olmadan UNKNOWN olarak ulaşır. @GrpcAdvice, @ControllerAdvice gibi çalışır:

```

@GrpcAdvice
public class CatalogExceptionAdvice { // public: Spring gRPC calls the
    ↪ handlers reflectively

    @GrpcExceptionHandler
    public Status notFound(BookNotFoundException e) {
        return Status.NOT_FOUND.withDescription(e.getMessage());
    }

    @GrpcExceptionHandler
    public Status invalid(IllegalArgumentException e) {
        return Status.INVALID_ARGUMENT.withDescription(e.getMessage());
    }
}

```

! Uyarı

Advice sınıfı ve handler metotları public olmalıdır. Spring gRPC onları reflection ile çağırır. Package-private bir handler IllegalAccessException ile başarısız olur ve istemci UNKNOWN alır.

Blocking stub bir StatusRuntimeException fırlatır. Onun getStatus()’u kodu ve açıklamayı içerir.

3.5 Interceptor’lar

Bir sunucu interceptor’ı, bir servlet filtresi gibi her çağrıyı sarar. Bu interceptor metodu, son durumu ve süreyi loglar:

```

@Component
@GlobalServerInterceptor // applies to all gRPC services
↪ of this server
@Order(Ordered.HIGHEST_PRECEDENCE) // outermost: it also sees
↪ calls closed by the error mapping
public class CallLoggingInterceptor implements ServerInterceptor {

    private static final Logger log =
        ↪ LoggerFactory.getLogger(CallLoggingInterceptor.class);

    private final List<String> recentCalls = new CopyOnWriteArrayList<>();

    @Override
    public <Q, R> ServerCall.Listener<Q> interceptCall(ServerCall<Q, R> call, Metadata
        ↪ headers,
                                                    ServerCallHandler<Q, R> next) {

        long start = System.nanoTime();
        String method = call.getMethodDescriptor().getFullMethodName();
        ServerCall<Q, R> timedCall = new SimpleForwardingServerCall<>(call) {
            @Override
            public void close(Status status, Metadata trailers) { // called once, when
                ↪ the call ends
                    log.info("{} {} in {} ms", method, status.getStatusCode(), (System.nanoTime() -
↪ start) / 1_000_000);
                    recentCalls.add(method + " " + status.getStatusCode());
                    super.close(status, trailers);
                }
            };
            return next.startCall(timedCall, headers);
        }
    }
}

```

@GlobalServerInterceptor onu tüm servislere uygular. @Order(HIGHEST_PRECEDENCE) onu en dıştaki interceptor yapar, böylece hata eşlemesinin kapattığı çağrılar da görür. Tipik interceptor'lar kimlik doğrulamayı kontrol eder, trace ID'leri ekler veya metrikleri ölçer.

3.6 İstemciler ve Deadline'lar

@ImportGrpcClients bir **mantıksal kanal** için stub bean'leri oluşturur. Gerçek adres yapılandırmasıdır:

```

@Configuration(proxyBeanMethods = false)
@ImportGrpcClients(target = "catalog", types = {
    BookCatalogGrpc.BookCatalogBlockingStub.class,
    ↪ BookCatalogGrpc.BookCatalogStub.class})
class ClientConfiguration {

```

```

}

grpc:
  server:
    port: 9090 # the default; gRPC runs on its own Netty
    ↪ server (HTTP/2)
  client:
    channel:
      catalog: # the logical name used by @ImportGrpcClients
        target: static://localhost:9090 # here the application calls itself (see
    ↪ LessonTour)
      default:
        deadline: 2s # every call without its own deadline
    ↪ gives up after 2 s

```

Bir **deadline**, istemcinin vazgeçtiği zaman noktasıdır. Çağrıyla birlikte sunucuya gider, sunucu da üzerindeki işi durdurabilir. Deadline olmadan, takılı kalmış bir sunucuya yapılan çağrı sonsuza kadar bekleyebilir:

```

@Component
public class CatalogClient {

    private final BookCatalogGrpc.BookCatalogBlockingStub catalog;

    CatalogClient(BookCatalogGrpc.BookCatalogBlockingStub catalog) { // imported by
    ↪ @ImportGrpcClients
        this.catalog = catalog;
    }

    public String titleOf(String isbn, Duration deadline) {
        return catalog.withDeadlineAfter(deadline.toMillis(), TimeUnit.MILLISECONDS) //
        ↪ give up after this time
            .getBook(GetBookRequest.newBuilder().setIsbn(isbn).build())
            .getTitle();
    }
}

```

```

@Test
void aTooShortDeadlineEndsTheCall() {
    assertThatThrownBy(() -> client.titleOf("9780134685991", Duration.ofMillis(100)))
    ↪ // server needs 300 ms
        .isInstanceOfSatisfying(StatusRuntimeException.class,
            e ->
    ↪ assertThat(e.getStatus().getCode()).isEqualTo(Status.Code.DEADLINE_EXCEEDED));
}

```

★ İpucu

Her kanal için varsayılan bir deadline (`default.deadline`), bir kullanıcının istek yolundaki çağrılar için daha kısa bir deadline ayarlayın. Bir deadline mutlakdır: Başka servisleri çağırarak bir servis kalan süreyi aktarır.

3.7 Health, Reflection ve Testler

Classpath'te `io.grpc:grpc-services` ile Boot iki standart servis kaydeder:

- **Health** (`grpc.health.v1.Health`): Sunucunun SERVING bildirip bildirmediğine Boot'un health indicator'ları karar verir. Kubernetes onu bir gRPC probe'u olarak kullanabilir (modül 22).
- **Reflection**: `grpcurl` gibi istemciler servisleri listeleyebilir ve tanımlarını indirebilir.

@AutoConfigureTestGrpcTransport her kanalı ve sunucuyu **in-process** taşımayla değiştirir. Testler hiçbir ağ portuna ihtiyaç duymaz ve hızlıdır, yine de gerçek stub'lardan, serileştirmeden ve interceptor'lardan geçer:

```
@SpringBootTest(properties = "bookstore.tour.enabled=false")
@AutoConfigureTestGrpcTransport // in-process channels: fast, no
↪ network port
class BookCatalogServiceTest {

    @Autowired
    BookCatalogGrpc.BookCatalogBlockingStub catalog; // the application's stub
    ↪ (ClientConfiguration), now in-process

    @Autowired
    BookCatalogGrpc.BookCatalogStub asyncCatalog; // for client and bidirectional
    ↪ streaming

    @Test
    void unaryCall() {
        Book book = cata-
        ↪ log.getBook(GetBookRequest.newBuilder().setIsbn("9780134685991").build());

        assertThat(book.getTitle()).isEqualTo("Effective Java");
        assertThat(book.getPriceCents()).isEqualTo(8990);
    }
}
```

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Var olan bir alanın numarasını veya tipini asla değiştirmeyin ve silinmiş bir alanın numarasını asla yeniden kullanmayın (onu reserved olarak işaretleyin). Numara, kabloda giden tek şeydir: Eski istemciler hiçbir hata olmadan yanlış veri okurdu.

- **Yapın:** Yeni alanları yeni numaralarla ekleyin. Eski istemciler bilinmeyen alanları yok sayar ve yeni kod eski mesajlarda varsayılan değerleri (0, "") görür.
- **Yapmayın:** Exception'ların istemcilere UNKNOWN olarak ulaşmasına izin vermeyin. Onları istemcinin üzerine hareket edebileceği durum kodlarına eşleyin.
- **Yapın:** Her çağrıya bir deadline koyun ve DEADLINE_EXCEEDED ile UNAVAILABLE'ı ele alın (yeniden deneme veya yedek davranış).
- **Yapmayın:** Çift yönlü bir akışın stream observer'ını uzun süre bloklamayın. Cevap verin veya işi başka bir thread'e verin.
- **Yapın:** Paketi sürümlendirin (bookstore.catalog.v1). Kırıcı bir değişiklik v2 olur ve ikisi bir süre yan yana çalışır.

5. Özet

- .proto dosyası sözleşmedir. Maven eklentisi mesajları ve stub'ları üretir ve alan numaraları kablo formatını tanımlar.
- @GrpcService bean'leri unary ve streaming metotları StreamObserver ile gerçekleştirir.
- @GrpcAdvice exception'ları durum kodlarına eşler ve @GlobalServerInterceptor kesilen davranışlar ekler.
- @ImportGrpcClients mantıksal kanallar için stub'lar oluşturur. Deadline'lar çağırانları yavaş sunuculardan korur.
- Health ve reflection grpc-services ile gelir ve @AutoConfigureTestGrpcTransport her şeyi in-process test eder.
- REST public ve tarayıcı API'lerine, gRPC iç, tipli, streaming servisler arası çağrılara uyar.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — gRPC](#)
- [Spring gRPC Reference](#)
- [gRPC — Core concepts](#) · [Status codes](#) · [Deadlines](#)
- [Protocol Buffers — Language Guide \(proto3\)](#) · [Updating a message type](#)